

Practical: 2	Implementation and Time analysis of linear and binary search algorithm
07/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:

Algorithm:

1) Algorithm for Linear Search:-
 Step 1: START
 Step 2: Read the number of element n
 Step 3: Read the array element arr[0] to arr[n-1]
 Step 4: Read the element to be searched key
 Step 5: Set i=0
 Step 6: Repeat While i < n
 If arr[i] == Key, go to Step 7
 Otherwise, increment i = i+1
 Step 7: Display "Element Found" and its position
 Step 8: If i == n, display "Element not found"
 Step 9: Stop

2) Algorithm for Binary Search:-
 Step 1: Start
 Step 2: Read the number of element n
 Step 3: Read the sorted array elements
 Step 4: Read the element to be searched key
 Step 5: Set low=0 and high = n-1
 Step 6: Repeat While low <= high:
 • Calculate mid = low + (high-low)/2
 • If arr[mid] == Key, go to Step 7
 • If arr[mid] < Key set low = mid+1
 • Otherwise set high = mid-1
 Step 7: Display "Element Found" and its position
 Step 8: If low > high display "Element not found"
 Step 9: Stop

Linear Search Time-Complexity:-
 for (int i=0; i<n; i++)
 if (arr[i] == Key)
 return i;
 return -1;
 $T(n) = H(n+1) + n + n + 1 + 1$
 $T(n) = 3n + 4$
 Ignoring constants
 $T(n) = O(n)$

Binary Search Time-Complexity:-
 low = 0;
 high = n-1;
 while (low <= high)
 mid = (low+high)/2
 if (arr[mid] == Key) → log₂n
 return mid;
 else if (arr[mid] < Key) → log₂n
 low = mid+1
 else
 high = mid-1
 $T(n) = 1 + 1 + (\log_2 n + 1) + \log_2 n + 1 + 1$
 $T(n) = 5 \log_2 n + 3$
 Ignoring constants
 $T(n) = O(\log n)$

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

// Linear Search
int linearSearch(int arr[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

// Binary Search
int binarySearch(int arr[], int n, int key)
```

```
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int n, key;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;

    // Linear Search
    int linearResult = linearSearch(arr, n, key);

    if (linearResult != -1)
        cout << "Linear Search: Element found at index "
              << linearResult << endl;
    else
        cout << "Linear Search: Element not found" << endl;

    // Sort array for Binary Search
    sort(arr, arr + n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;

    // Binary Search
    int binaryResult = binarySearch(arr, n, key);

    if (binaryResult != -1)
        cout << "Binary Search: Element found at index "
              << binaryResult << endl;
    else
        cout << "Binary Search: Element not found" << endl;

    return 0;
}
```

}

Output:

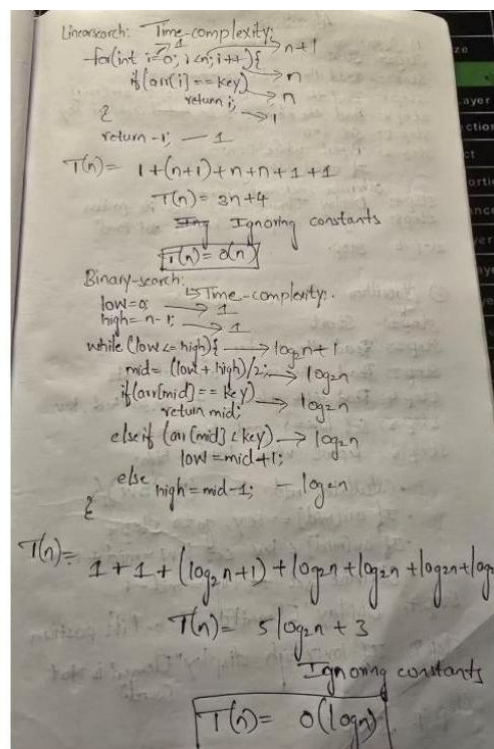
```

Output
Enter number of elements: 5
Enter elements: 40 10 30 20 50
Enter element to search: 20
Linear Search: Element found at index 3
Sorted array: 10 20 30 40 50
Binary Search: Element found at index 1

=== Code Execution Successful ===

```

Time Complexity:



Conclusion:

From this implementation, I understood that the searching method depends on the type of data. Linear Search is simple and checks elements one by one, while Binary Search is faster because it repeatedly divides the sorted array into halves.